

# Assignment 2: Stream Processing with Kafka Streams

Deadline: 01.06.2026, 23:59

Group 1, Team Members:

Michael Schütz, Gianluca Ielpo, Eva Amromin

## Contents

|                                           |    |
|-------------------------------------------|----|
| 1. Changes to Assignment 1 .....          | 2  |
| 2. Project Overview .....                 | 2  |
| 3. Stream Processing Implementation ..... | 4  |
| 3.1. Stateless Operations (Week 8) .....  | 4  |
| 3.2. KStream and KTable (Week 9) .....    | 5  |
| 3.3. Joins Across Streams (Week 9) .....  | 5  |
| 3.4. Interactive Queries (Week 9) .....   | 5  |
| 3.5. Windowed Operations (Week 10) .....  | 6  |
| 3.6. Serialization .....                  | 6  |
| 4. Topology Descriptions .....            | 6  |
| 4.1. BlockColorTopology .....             | 7  |
| 4.2. MoveBlockTopology .....              | 9  |
| 5. ADRs .....                             | 10 |
| 6. Reflections .....                      | 10 |
| 6.1. What went well .....                 | 10 |
| 6.2. Challenges .....                     | 10 |
| 6.3. What we would improve .....          | 10 |
| 7. Repository .....                       | 12 |
| 8. Contributions .....                    | 13 |

# 1. Changes to Assignment 1

Since the hand-in of Assignment 1, the documentation for Exercise 5 has been updated. ADR 0010 was added to document the decisions around communication protocols. The flow-success sequence diagram was extended for clarity, accompanied by additional explanatory text.

Three additional ADRs were added to document architectural decisions that were implicit in the Assignment 1 implementation: ADR 0014 (Hexagonal Architecture for Order and Factory Services), ADR 0015 (Embedded H2 Database for Process Engine Persistence), and ADR 0016 (Correlation ID Strategy).

## 2. Project Overview

KAFKEA is an event-driven manufacturing system for custom furniture orders. In the first assignment, we implemented the core order processing and factory execution logic using Camunda BPMN processes. A customer can place an order via a Camunda form. The Order service then orchestrates the end-to-end process, with the Factory service operating the Dobot Magician. For a full description of the project see the previous reports (Exercises 1-2, 3-4, and 5).

This report covers the stream processing layer added for Assignment 2, implemented in the `kafka-streams` service. The service uses Kafka Streams to process sensor data in order to restock the inventory: it detects blocks, classifies their color, and sends commands to the conveyor belt.

The new service resides in the inventory bounded context. Because the inventory is now being restocked by the Dobot Magician and not by hand, the robot control is now a shared kernel between the inventory and factory contexts.

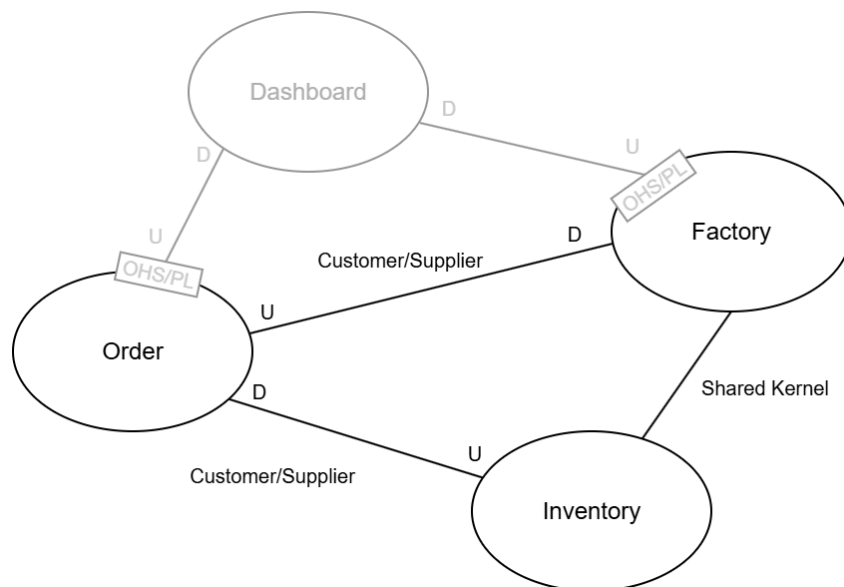


Figure 1: Updated context map

To simplify development, the simulator was enhanced to produce synthetic sensor data that mimics the real factory layout. The left conveyor sensor produces distance readings that indicate when a block is present, while the color sensor produces RGB readings that are classified to determine the block's color.

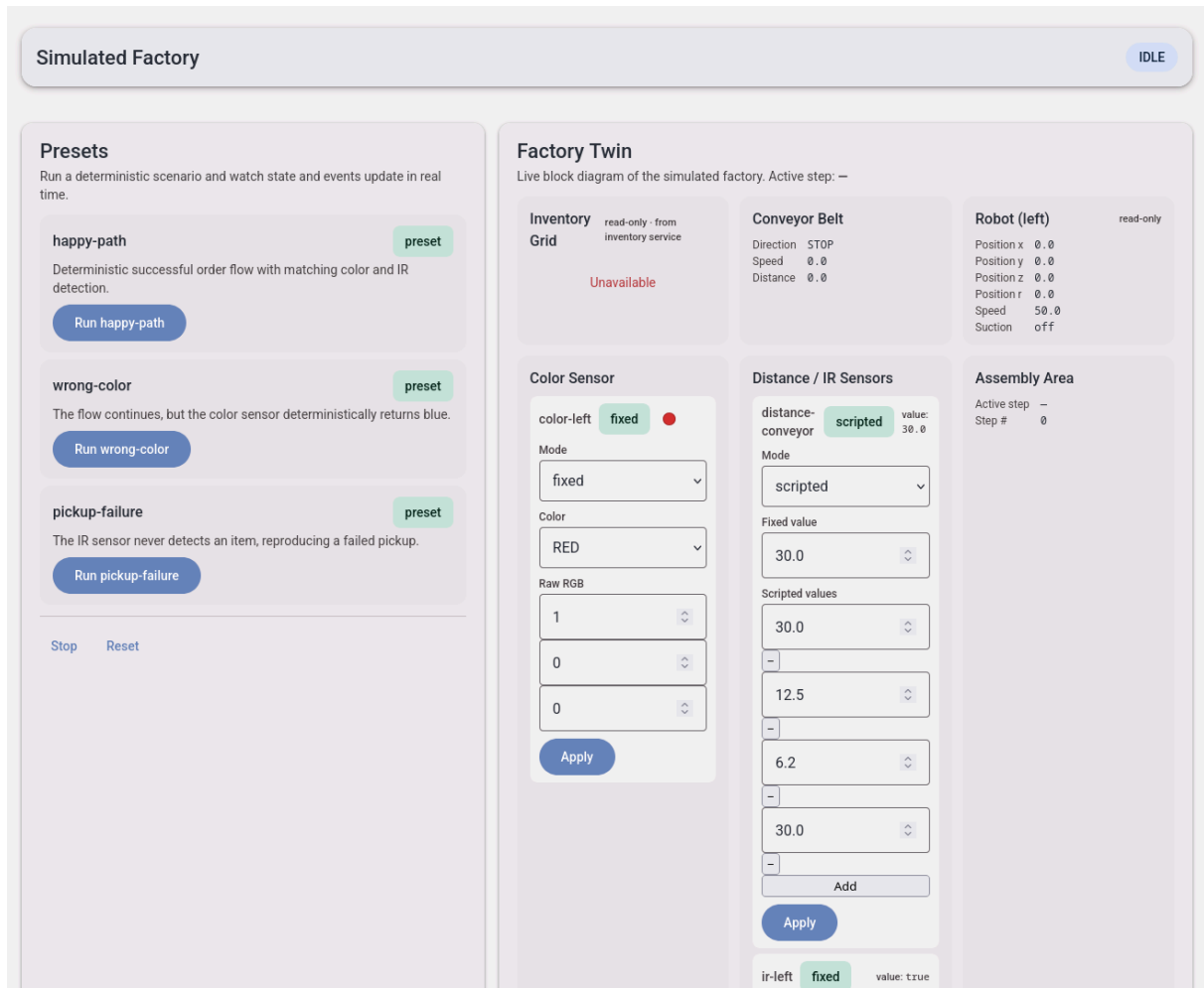


Figure 2: Simulated factory layout showing color sensor and distance sensor

The following table lists the Kafka topics consumed and produced by the kafka-streams service.

| Topic                        | Direction | Purpose                                                                                                                                             |
|------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| sensor.distance.raw.v1       | in        | Raw distance readings from the left conveyor distance sensor (forwarded from MQTT)                                                                  |
| sensor.color.raw.v1          | in        | Raw RGB readings from the color sensor (forwarded from MQTT)                                                                                        |
| sensor.block-present.v1      | internal  | Filtered block-present readings forwarded from BlockColorTopology to MoveBlockTopology                                                              |
| sensor.block-detected.v1     | internal  | Block-detected events emitted by the wall-clock inactivity processor when a block has passed the distance sensor; diagnostic and demo purposes only |
| color.classified.v1          | internal  | Classified color readings produced by the color branch of BlockColorTopology; diagnostic and demo purposes only                                     |
| inventory.blocks.v1          | out       | Enriched block and color events; also queryable via GET /inventory                                                                                  |
| control.conveyor.commands.v1 | out       | Conveyor commands emitted by MoveBlockTopology                                                                                                      |

Table 1: Kafka topics consumed and produced by the kafka-streams service

### 3. Stream Processing Implementation

The following sections map the implemented stream processing patterns to the relevant lecture concepts.

#### 3.1. Stateless Operations (Week 8)

Stateless operations transform or filter individual records without requiring any shared state between records.

**Filter** is applied to discard distance readings that do not indicate a block is present. The threshold is  $\text{distance} < 25.0$  (cm), matching the physical range of the conveyor sensors. It is also used to discard invalid color readings.

**Translate (Map)** is used to translate domain events, for example, converting raw RGB values into a `BlockColor` enum.

**Translate (SelectKey)** re-keys streams to the fixed partition keys required for downstream joins and aggregations. For instance, all distance readings are re-keyed to "distance-sensor", and both the block-detected and color streams are re-keyed to "sliding-window-join" before the join.

**Branch** is implemented implicitly: `BlockColorTopology` publishes the `blockPresentStream` both to the `sensor.block-present.v1` topic (consumed by `MoveBlockTopology`) and processes it internally for block detection. This separation of concerns mirrors the branch pattern.

**Enrich** is realised by the stream-stream join in `BlockColorTopology`, which enriches a bare `BlockDetectedEvent` (containing only a UUID and timestamp) with color information from the color sensor stream.

### 3.2. KStream and KTable (Week 9)

The topologies use both the streaming and table abstractions.

`KStream` is the primary abstraction for unbounded sensor data: distance readings, color readings, and intermediate events all flow as streams. After the join and reduce, the enriched inventory is materialised into a `KTable` (`inventory-store`), representing the current state of all detected blocks. The table is then converted back to a stream to publish to `inventory.blocks.v1`.

The aggregation state store in `MoveBlockTopology` (`move-block-edge-store`) is also `KTable`-backed, persisting the last-seen timestamp, rising-edge flag, and edge timestamp per sensor key.

### 3.3. Joins Across Streams (Week 9)

`BlockColorTopology` joins two independent sensor streams: the block-detected stream, emitting one event per detected block keyed to `"sliding-window-join"`, and the classified color stream, carrying continuous RGB readings classified to a `BlockColor`, also keyed to `"sliding-window-join"`.

A **stream-stream sliding window join** with a 10-second window (1-second grace) is used, because the block and color readings are produced at different rates and cannot be perfectly aligned in time. Each block is placed on the color sensor only after the detection event is emitted.

Both streams are explicitly repartitioned before the join to ensure matching keys land on the same task.

### 3.4. Interactive Queries (Week 9)

The `inventory-store` `KTable` is exposed as a queryable state store. The `InventoryQueryController` REST endpoint allows any client to query the current inventory state directly from the Kafka Streams instance, without reading from Kafka topics. `GET /inventory` returns all detected blocks as a cubeId-to-color map. `GET /inventory/{cubeId}` returns the entry for a specific cube.

The controller uses `StoreQueryParameters.fromNameAndType` and `QueryableStoreTypes.keyValueStore()` to access the read-only key-value store at runtime.

### 3.5. Windowed Operations (Week 10)

**Sliding window join:** The stream-stream join in `BlockColorTopology` uses `JoinWindows.ofTimeDifferenceAndGrace(Duration.ofSeconds(10), Duration.ofSeconds(1))`. This is a sliding window over event time, allowing block and color events that arrive within 10 seconds of each other to be joined. Due to the events not being generated on the same schedule, the sliding window join is a good solution for matching events from both streams.

**Wall-clock inactivity window (custom processor):** Block detection in `BlockColorTopology` relies on a custom `BlockInactivityProcessor`. A wall-clock punctuation triggers every 200 ms and emits a `BlockDetectedEvent` when a sensor key has not received events for more than 3 seconds. This implements session-window semantics driven by wall-clock time rather than stream time, which is necessary because physical blocks create a continuous stream of distance readings with no natural end marker. A standard Kafka Streams session window was not used because it advances only when new records arrive (see ADR 0012).

**Rising-edge aggregation:** `MoveBlockTopology` uses a stateful aggregate with a 2-second gap threshold to detect the first reading of a new block. This mirrors a session-window boundary condition: after a period of inactivity exceeding the gap threshold, the next record is treated as the start of a new session and triggers exactly one command to move the conveyor.

Tumbling and hopping windows are not used in the current implementation. The use cases in KAFKEA (block detection and color join) do not require fixed-interval aggregation, making sliding and session-style windows more appropriate. See the Reflections section for a discussion of where a tumbling window could be added as an additional analytical step.

### 3.6. Serialization

All events are serialized and deserialized using a custom `JsonSerde<T>` built on Jackson. This keeps the setup self-contained without requiring a schema registry.

## 4. Topology Descriptions

This section describes the two Kafka Streams topologies implemented in the `kafka-streams` service, including their processing steps and design justifications.

## 4.1. BlockColorTopology

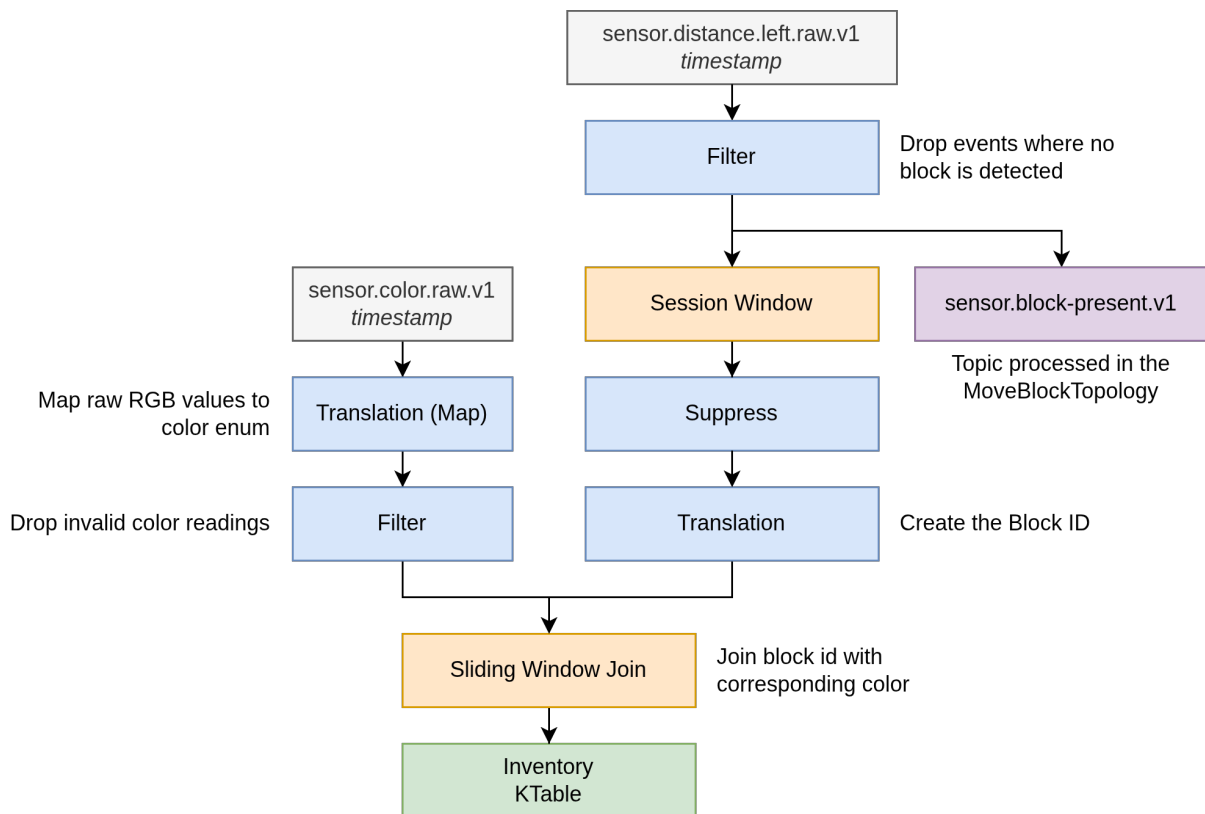


Figure 3: BlockColorTopology for block detection and color classification

BlockColorTopology is the more complex topology. It detects block presence on the conveyor (distance branch), detects when a block has passed (inactivity processor), classifies the block's color (color branch), joins the two observations to produce a BlockColorEvent, and maintains a deduplicated inventory of all detected blocks.

The topology reads from two source topics: `sensor.distance.raw.v1` (continuous distance readings from the left distance sensor) and `sensor.color.raw.v1` (continuous RGB readings from the color sensor). Both streams are filtered and rekeyed, then joined on the common key "sliding-window-join" within a 10-second sliding window. The join output is reduced per `cubeId` to keep only the first color reading per block, ensuring idempotent inventory updates. The final result is published to `inventory.blocks.v1` and stored in the `inventory-store` KTable.

Additionally, every raw distance reading that indicates a block is present is forwarded to `sensor.block-present.v1`, where it is consumed by MoveBlockTopology.

### Processing steps:

Distance branch:

1. Source `sensor.distance.raw.v1` → `KStream<String, DistanceEvent>`.
2. Filter: `distance < 25.0 cm` → block is present.
3. SelectKey → "distance-sensor" (fixed key for consistent partitioning).
4. Publish block-present readings to `sensor.block-present.v1` (consumed by MoveBlockTopology).

5. Custom `BlockInactivityProcessor` backed by the block-activity-store KV state store: records the last-seen wall-clock timestamp per key. A wall-clock punctuation triggers every 200 ms and emits a `BlockDetectedEvent(cubeId=UUID, timestamp)` when the key has been quiet for at least 3 seconds.
6. `SelectKey` → "sliding-window-join" + explicit repartition (ensures join tasks see matching keys).

Color branch:

1. Source `sensor.color.raw.v1` → `KStream<String, ColorEvent>`.
2. `MapValues`: `classify (r, g, b) → BlockColor enum (RED / GREEN / BLUE / YELLOW, or UNKNOWN if below brightness threshold)`.
3. `Filter`: discard UNKNOWN readings.
4. `SelectKey` → "sliding-window-join" + explicit repartition.
5. Publish classified color to `color.classified.v1`.

Join and materialization:

1. Stream-stream sliding-window join (10 s window, 1 s grace) → `BlockColorEvent(cubeId, color, timestamp)`.
2. `SelectKey` → `cubeId` + `groupByKey` + `reduce(first)` materialised as `KTable "inventory-store"`: keeps only the first confirmed color per cube, ensuring idempotent inventory entries regardless of duplicate join outputs.
3. `toStream` → publish to `inventory.blocks.v1`.

**Justification:** The wall-clock inactivity processor was chosen over a session window because Kafka Streams session windows advance only with new records, leading to the window closing only when the next block is placed for processing. Wall-clock punctuation ensures block detection fires when the current block leaves the sensor without having to wait.

The sliding window join was chosen over a table join because color events are continuous and not keyed to a specific block identity. Temporal proximity is the only way to associate a color reading with a block.

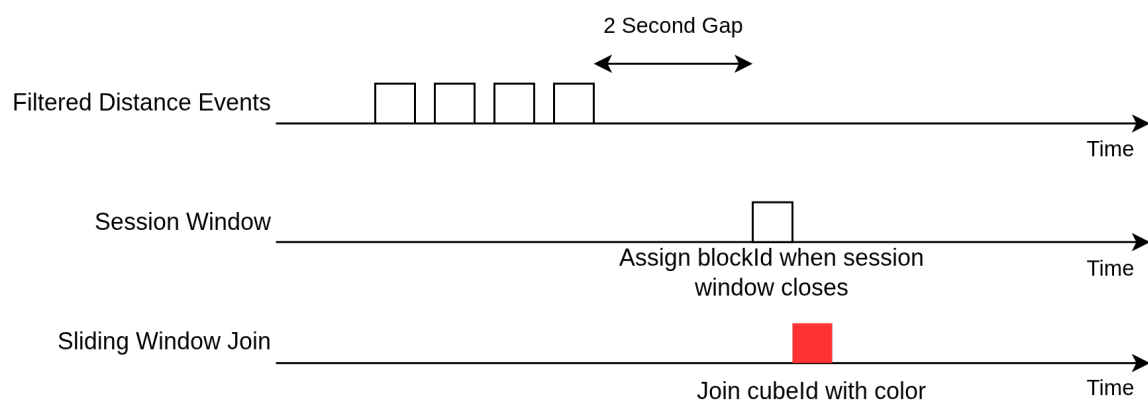


Figure 4: BlockColorTopology event timeline



## 4.2. MoveBlockTopology

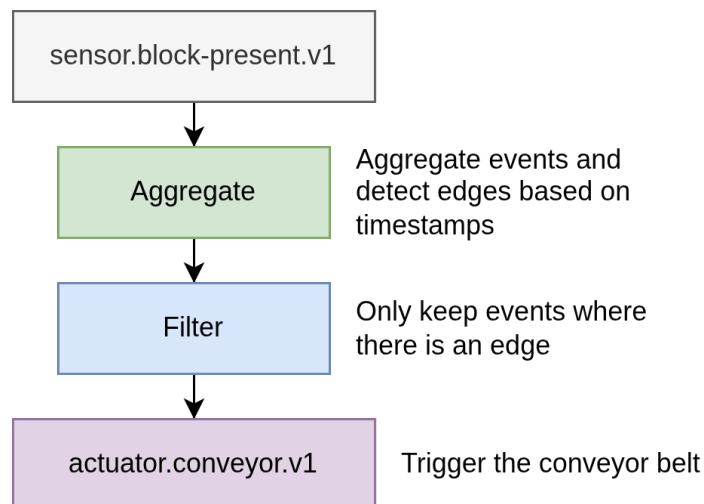


Figure 5: MoveBlockTopology to move block from drop-zone to color sensor

MoveBlockTopology commands the conveyor belt to move the block from the distance sensor to the color sensor. It reads from `sensor.block-present.v1` (pre-filtered readings published by BlockColorTopology) and applies rising-edge detection via a stateful aggregate: a `ConveyorCommandEvent("MOVE")` is emitted exactly once per block arrival, defined as the first reading after a 2-second absence gap.

### Processing steps:

1. Source `sensor.block-present.v1` → `KStream<String, DistanceEvent>` (already keyed to "distance-sensor" by BlockColorTopology).
2. `GroupByKey` + `aggregate` → `KTable "move-block-edge-store"` of `BlockPresenceState(lastSeenTimestamp, risingEdge, edgeTimestamp)`. A rising edge is flagged when  $\text{currentTimestamp} - \text{lastSeenTimestamp} > 2 \text{ s}$ .
3. `toStream` + `filter`: keep only states where `risingEdge` is true.
4. `MapValues` → `BlockMoveTriggerEvent(UUID, edgeTimestamp)`.
5. `MapValues` → `ConveyorCommandEvent("MOVE", triggerId, timestamp)`.
6. Publish to `control.conveyor.commands.v1`.

**Justification:** Consuming from `sensor.block-present.v1` rather than re-reading the raw distance topic avoids duplicating the filter logic and keeps the topology simple. The rising-edge aggregation prevents sending multiple `MOVE` commands for the same block.

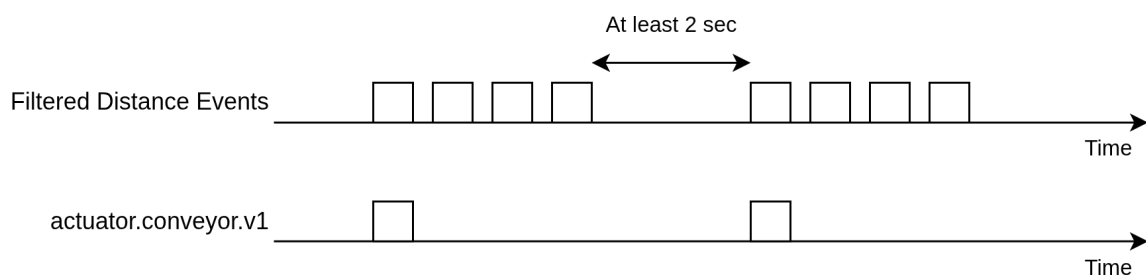


Figure 6: MoveBlockTopology: event timeline

## 5. ADRs

The following ADRs are related to this assignment:

- ADR 0011: Rising-Edge Detection for Sensor Triggers.
- ADR 0012: Wall-Clock Punctuation for Block Detection.
- ADR 0013: JSON Serialization over Avro.

## 6. Reflections

This section summarises what worked well during the implementation, the challenges we encountered, and what we would improve in a future iteration.

### 6.1. What went well

The decomposition into two topologies made it straightforward to work on each stream independently. `BlockColorTopology` handles block identity and color classification. `MoveBlockTopology` consumes pre-processed events and applies simple rising-edge logic. The separation of concerns meant changes to inactivity detection (for example, adjusting the 2-second gap) had no impact on the conveyor topology.

Materialising the inventory as a `KTable` with `reduce(first)` gave us idempotent inventory updates without any additional deduplication step. Kafka's at-least-once delivery combined with the first-wins reduction means that duplicate join outputs (which are expected with sliding windows) do not corrupt the inventory state.

The simulator made end-to-end testing possible without physical hardware. Being able to replay synthetic sensor sequences locally significantly reduced iteration time when tuning the inactivity and rising-edge thresholds.

### 6.2. Challenges

The wall-clock punctuation in `BlockColorTopology` required stepping outside the Kafka Streams DSL and manually managing a state store, making the implementation more verbose than a DSL session window would have been. It was also necessary to be careful about the order of state store registration and topology wiring to avoid startup errors.

Getting the sliding-window join to produce correct results required explicit repartitioning of both input streams before the join. Without it, records with the same logical key could land on different tasks and never be joined. This behaviour is not immediately obvious from the Kafka Streams documentation.

The RGB color classification thresholds are heuristic. Calibrating them against real sensor readings requires iterating with the physical hardware. The simulator uses ideal values that do not fully reflect real conditions.

### 6.3. What we would improve

**Avro serialisation.** The current `JsonSerde<T>` backed by Jackson is self-contained but provides no schema evolution guarantees. Replacing it with Avro and a schema

registry would enforce compatibility between producers and consumers and reduce payload size on the wire.

**Explicit windowed aggregation.** The windowed operations in this assignment are covered by the sliding-window join and the rising-edge aggregate. Adding an explicit tumbling or hopping window aggregation (for example, counting blocks per color per minute for restocking analytics) would more directly illustrate the windowed-operations pattern from the lecture.

**Integration tests.** No `TopologyTestDriver` tests exist for any of the two topologies. Adding unit-level topology tests would make regressions immediately visible without needing to run the full Kafka stack.

**Configurable color thresholds.** The RGB classification thresholds in `BlockColor.from()` are currently hardcoded constants. Externalising them into `application.yml` would simplify calibration for different lighting conditions or sensor hardware without requiring a recompile.

**Simulated factory.** The simulated factory is a focused digital twin of only the restocking process, in a future iteration it would be interesting to implement a complete twin of the KAFKEA application.

## 7. Repository

Source code: [github.com/ielpo/edpo-project-group1](https://github.com/ielpo/edpo-project-group1)

The kafka-streams service is located at `services/kafka-streams/`.

The final release is available at [github.com/ielpo/edpo-project-group1/releases/tag/assignment-2](https://github.com/ielpo/edpo-project-group1/releases/tag/assignment-2)

## 8. Contributions

| Person   | Tasks                                                                                      |
|----------|--------------------------------------------------------------------------------------------|
| Michael  | Building topologies, Validation against lecture content, Documentation, ADRs, Presentation |
| Eva      | Building topologies, Topology implementation, Presentation                                 |
| Gianluca | Building topologies, Topology implementation, Simulator, Presentation                      |